

Índice da documentação

Agenda ao Ar Livre | Entrega 1 | Luca Romariz e Miguel Silva

Leitura sugerida

1. Documento de Visão: visao/visao.md e visao/visao.pdf.
2. Casos de uso: modelagem/casos-de-uso/especificacoes-casos-de-uso.md e PDF de mesmo nome.
3. Arquitetura: arquitetura/arquitetura.md e arquitetura/arquitetura.pdf.
4. Classes: modelagem/classes/diagrama-de-classes.md e PDF de mesmo nome.
5. Dados: modelagem/banco-de-dados/diagrama-er.md e modelo-logico.md, com PDFs correspondentes.
6. APIs: api/contrato-api.md, api/openapi.json e api/integracao-externa.md, com PDFs textuais.
7. Protótipos: prototipos/index.html, prototipos/telas.md e identidade-e-prototipos.md/PDF, com capturas das telas.
8. Planejamento: planejamento/planejamento.md/PDF, backlog.csv e submissao.md.

Diagramas UML/ER possuem arquivos .drawio editáveis e .svg exportados ao lado de seus PDFs. A fonte das tabelas também está em modelo.json, com esquema.sql ilustrativo. Os arquivos PDF institucionais foram preenchidos nos mesmos caminhos do template.

Situação real

Material de fase 1 preparado com auxílio de IA, sujeito à revisão da dupla. CEUB, ADS, turma A; prazo informado: 05/10/2026 às 08h (horário de Brasília). Backend, hospedagem, testes funcionais e SAST/DAST pertencem à fase 2 e ainda não estão implementados. Revisão da dupla, evidências individuais de contribuição e publicação no GitHub permanecem pendentes.

Pacote consolidado

entrega-1.pdf reúne os documentos em ordem de leitura. Os arquivos separados e editáveis permanecem disponíveis no repositório.

Documento de Visão

Agenda ao Ar Livre | Fase 1 | Versão 0.1 | 04/10/2026

1. Identificação e contexto

Integrantes: Luca Romariz e Miguel Silva. Instituição: CEUB. Curso: Análise e Desenvolvimento de Sistemas (ADS). Turma: A. Disciplina: Desenvolvimento Web. Professor: Felipe Pires Ferreira. Prazo da Entrega 1 informado pelo grupo: 05/10/2026 às 08h (horário de Brasília). Matrículas e prazo da fase 2 não informados.

Pequenos grupos organizam caminhadas, treinos e encontros em espaços públicos usando mensagens dispersas. Participantes podem perder informações sobre horários, endereço, vagas e cancelamentos. Organizadores precisam consolidar inscrições e consultar a previsão em outro serviço.

2. Problema e justificativa

A ausência de um registro central dificulta saber qual atividade está confirmada, quantas vagas existem e qual previsão corresponde à sua localização e data. A proposta reúne esses dados para reduzir consultas repetidas e apoiar decisões do organizador. O problema é uma hipótese de projeto, ainda sem pesquisa de campo; não se afirma validação com usuários reais.

3. Objetivos

Objetivo geral: planejar e implementar uma aplicação web Django que centralize atividades gratuitas ao ar livre, inscrições e previsão meteorológica.

- Permitir ao organizador manter locais e atividades de sua autoria.
- Permitir ao público pesquisar e consultar atividades publicadas.
- Controlar vagas e inscrições sem duplicidade nem exceder a capacidade.
- Apresentar previsão diária do local e data, sem prometer previsão fora do horizonte disponível.
- Gerar indicadores filtrados e exportar relatório CSV.
- Oferecer API REST de consulta pública e documentada.

4. Público-alvo e stakeholders

Envolvido	Interesse / responsabilidade
Visitante	Descobrir atividades e consultar informações públicas.
Participante	Inscrever-se, consultar inscrições e cancelá-las.
Organizador	Publicar suas atividades, gerenciar vagas e obter relatórios.
Luca Romariz e Miguel Silva	Desenvolver, revisar e apresentar a solução.
Professor	Avaliar artefatos, consistência e participação individual.
Open-Meteo	Prover os dados meteorológicos sob suas condições de uso.

Participante e organizador são papéis de uso da mesma conta: qualquer usuário autenticado pode organizar e participar. Não há escolha de perfil que conceda acesso a dados de outro organizador. A administração técnica usa Django Admin e não constitui uma funcionalidade de negócio adicional.

5. Escopo e requisitos funcionais

ID	Requisito e critério de aceite
RF01	Criar conta, entrar e sair. E-mail normalizado único; senha validada e armazenada com hash; falhas não revelam existência da conta.
RF02	Manter locais próprios com nome, endereço e coordenadas válidas. Impedir exclusão de local referenciado.
RF03	Criar, consultar, editar e excluir atividades próprias; exclusão física somente em rascunho e sem histórico de inscrição.
RF04	Buscar atividades por texto, período de início, cidade e categoria; exibir resultado vazio claramente.
RF05	Inscrever usuário em atividade publicada futura com vaga; impedir duplicidade e excesso em requisições concorrentes.
RF06	Consultar minhas inscrições e cancelar inscrição confirmada antes do início, liberando vaga.
RF07	Cancelar atividade própria futura, informar motivo e cancelar inscrições confirmadas na mesma transação.
RF08	Exibir previsão diária do local/data e fonte; mostrar indisponibilidade e fora de horizonte sem impedir outras funções.
RF09	Gerar relatório das próprias atividades por intervalo de início e situação; visualizar totais e exportar CSV equivalente.
RF10	Expor catálogo, detalhes, categorias e previsão via API REST pública v1, sem divulgar participantes, e-mails ou rascunhos.
RF11	Aplicar identidade visual e interface responsiva em telas essenciais.

6. Regras de negócio

- RN01: cada atividade tem exatamente um organizador, local e categoria. Apenas o dono pode alterá-la; o local escolhido deve pertencer a esse organizador.
- RN02: início futuro na criação/publicação; término posterior ao início e no mesmo dia no fuso America/Sao_Paulo. Capacidade inteira entre 1 e 500. Escopo geográfico inicial: locais atendidos nesse fuso.
- RN03: estados da atividade: rascunho, publicada, cancelada. Evento encerrado é uma condição derivada de término $\leq$ agora, não uma quarta situação persistida. Cancelada não reabre.
- RN04: publicar exige todos os campos válidos. Atividade publicada não volta a rascunho; exclusão física somente de rascunho sem qualquer inscrição. As demais são canceladas e mantidas no histórico.

- RN05: inscrição única por usuário/atividade. Confirmada ocupa uma vaga; cancelada não ocupa. Reinscrição reativa o mesmo registro se ainda elegível; o relatório não conta tentativas como pessoas diferentes.
- RN06: a criação/reativação da inscrição, cancelamento da atividade e alterações da capacidade usam transação e bloqueio da linha da atividade. A verificação de vagas ocorre dentro desse bloqueio.
- RN07: atividade já iniciada não permite novas inscrições nem cancelamentos ou alterações de negócio. Antes do início, capacidade nunca fica abaixo do total confirmado. Com qualquer histórico de inscrição, local, categoria, início e término ficam imutáveis; alteração substancial exige cancelar e criar nova atividade.
- RN08: cancelamento de atividade exige motivo de 10 a 500 caracteres. Motivo é público. Inscrições confirmadas mudam para cancelada com motivo atividade_cancelada. Cancelamentos individuais anteriores são preservados.
- RN09: alterar endereço/coordenadas de local com qualquer atividade publicada ou com inscrição é bloqueado; deve-se cadastrar outro local. Assim se preserva a localização histórica.
- RN10: o dono da atividade não se inscreve nela. Visitante só vê publicada/cancelada; rascunho é privado. Inscrições e relatórios são restritos ao respectivo usuário/dono.
- RN11: previsão é informação de apoio, nunca cancela automaticamente. Sem coordenadas válidas, o cadastro do local não é salvo. Sem previsão válida, mostrar estado textual sem inventar números.
- RN12: vagas = capacidade - inscrições confirmadas. Não persistir vagas, percentuais ou contadores derivados.

7. Fora do escopo

Pagamentos, ingressos, chat, mapas interativos, geolocalização do participante, WhatsApp, envio de e-mails, recuperação de senha por e-mail, recorrência, lista de espera, presença/check-in, calendário externo, alertas automáticos e previsões históricas. Coordenadas são informadas manualmente pelo organizador. Não há garantia de reserva física do espaço público.

8. Restrições e requisitos não funcionais

- RNF01: Python, Django, Django REST Framework e banco PostgreSQL; frontend com templates Django, HTML e CSS. Evitar SPA para reduzir complexidade de duas pessoas.
- RNF02: validação no servidor, autenticação de sessão, proteção CSRF em operações de escrita e autorização por proprietário.
- RNF03: publicação HTTPS na fase 2, DEBUG desativado, segredos por ambiente e cookies seguros em produção.
- RNF04: páginas usáveis a partir de 360 px, teclado, foco visível, labels e mensagens que não dependam só de cor.
- RNF05: meta de 2 s para consultas locais em base de demonstração de 1.000 atividades, desconsiderando rede externa; medir na fase 2. Serviço meteorológico com tempo total alvo de até 5 s e cache de 30 min.
- RNF06: dependências fixadas e migrations versionadas na fase 2; testes de fluxo, autorização e concorrência no PostgreSQL.

- RNF07: o catálogo não expõe dados pessoais de participantes. Usar dados fictícios para demonstração, sem coletar CPF, telefone ou localização pessoal.

9. Premissas e riscos

Premissas: equipe com duas pessoas; acesso ao GitHub; uso educacional não comercial; conexão disponível. Entrega 1 até 05/10/2026 às 08h (horário de Brasília); horizonte da fase 2 ainda a confirmar. As responsabilidades propostas dependem de aceite do grupo.

Risco	Impacto e resposta
API indisponível / limite	Preservar catálogo e inscrições; cache e mensagem explicativa; testar falhas simuladas.
Evento fora do horizonte	Mostrar que a previsão ainda não está disponível; não reutilizar outra data.
Escopo exceder o prazo	Priorizar RF01-RF10; manter exclusões; revisar backlog semanalmente.
Duas pessoas disputarem última vaga	Bloqueio transacional no PostgreSQL e teste concorrente.
Hospedagem interrompida	Definir provedor até marco M2 e verificar disponibilidade durante avaliação.
Documentos divergirem do código	Rastreabilidade por RF/UC e revisão a cada mudança.
Histórico concentrado em uma conta	Cada integrante revisa e altera seu trabalho com a própria conta; não simular commits.

10. Critérios de sucesso

Fase 1: nove produtos obrigatórios legíveis, diagramas editáveis e exportados, requisitos rastreados, template preservado e repositório acessível ao professor com participação real da dupla.

Fase 2: demonstração de cadastro, busca, inscrição, cancelamento, relatório e APIs; última vaga protegida em teste concorrente; pessoa A não altera dados de B; falha meteorológica não impede inscrição; aplicação pública HTTPS, testes e evidências SAST/DAST do próprio sistema. Nenhuma dessas evidências de implementação é declarada concluída nesta fase.

Especificações de casos de uso

Agenda ao Ar Livre | Fase 1 | 04/10/2026

Atores e convenções

Visitante consulta o catálogo e cria conta. Participante utiliza inscrições. Organizador mantém seus locais/atividades e relatórios. Participante e organizador são papéis simultâneos de uma conta. Open-Meteo é ator secundário; Cliente de API representa sistemas terceiros.

Autenticação é pré-condição dos fluxos protegidos, não um include repetido em cada operação. UC02 e UC11 incluem UC03 quando consultam o detalhe meteorológico (no caso da API, especificamente a rota de previsão). No diagrama, UC11 não possui include obrigatório, pois nem toda consulta de API pede previsão.

Diagrama UML

Diagrama ou prancha visual disponível ao final deste documento; fonte editável e SVG na mesma pasta.

Fonte editável: casos-de-uso.drawio. Elipses representam objetivos; linhas ligam atores; seta tracejada include indica comportamento obrigatório do detalhe UC02.

UC01 - Criar conta e autenticar-se

Atores: Visitante / usuário. **Requisito:** RF01. **Prioridade:** alta.

Objetivo: Criar conta e autenticar-se no contexto do Agenda ao Ar Livre.

Precondições: Conta ativa para login; para cadastro, e-mail ainda não utilizado.

Pós-condições de sucesso: Conta criada e/ou sessão iniciada; logout invalida a sessão.

Pós-condições de falha: nenhuma alteração parcial; mensagens preservam dados de formulário quando aplicável.

Fluxo principal

1. Informar nome, e-mail, senha e confirmação no cadastro.
2. Validar campos, normalizar e-mail e aplicar validadores de senha do Django.
3. Salvar conta com hash da senha e solicitar login.
4. Receber credenciais, validar conta ativa e iniciar sessão.
5. No logout, encerrar a sessão por POST com CSRF.

Fluxos alternativos

- Cadastro repetido: informar impossibilidade de concluir e orientar login sem divulgar detalhes da conta.
- Credenciais incorretas: mensagem genérica; limitar tentativas a 5 por minuto por origem e identificador.

Exceções

- Falha de persistência: rollback; não criar conta parcial.
- CSRF inválido em login/logout: rejeitar operação.

UC02 - Pesquisar e consultar atividades

Atores: Visitante, participante e organizador. **Requisito:** RF04. **Prioridade:** alta.

Objetivo: Pesquisar e consultar atividades no contexto do Agenda ao Ar Livre.

Precondições: Aplicação disponível.

Pós-condições de sucesso: Lista ou detalhe público apresentado sem alteração de dados.

Pós-condições de falha: nenhuma alteração parcial; mensagens preservam dados de formulário quando aplicável.

Fluxo principal

1. Informar texto, cidade, categoria e/ou datas de início.
2. Validar intervalo; buscar somente publicada/cancelada.
3. Paginar e ordenar por início e id.
4. Selecionar atividade e visualizar endereço, horário, situação e vagas.
5. Incluir UC03 para o bloco meteorológico do detalhe.

Fluxos alternativos

- Sem resultados: oferecer limpeza dos filtros.
- Rascunho: só o dono acessa pela área de gestão; catálogo público retorna não encontrado.
- Evento cancelado: mostrar motivo e desabilitar inscrição.

Exceções

- Período inválido: exibir erro e conservar filtros.
- Identificador ausente: retornar página não encontrada.

UC03 - Consultar previsão

Atores: Visitante, participante e organizador; secundário: Open-Meteo. **Requisito:** RF08. **Prioridade:** alta.

Objetivo: Consultar previsão no contexto do Agenda ao Ar Livre.

Precondições: Atividade acessível, local com coordenadas válidas.

Pós-condições de sucesso: Previsão diária ou estado de indisponibilidade apresentado; atividade inalterada.

Pós-condições de falha: nenhuma alteração parcial; mensagens preservam dados de formulário quando aplicável.

Fluxo principal

1. Converter início para data local America/Sao_Paulo.
2. Verificar horizonte e procurar cache por coordenadas/data/fuso.
3. Na ausência de cache válido, consultar Open-Meteo pelo backend.
4. Validar estrutura JSON, unidades, data e valores; normalizar resposta.
5. Exibir mínima, máxima, probabilidade máxima de precipitação, código traduzido, fonte e instante da consulta.

Fluxos alternativos

- Fora do horizonte: mostrar fora_do_horizonte sem solicitar API.
- Cache válido: reutilizar por até 30 minutos.
- Dado parcial/nulo: apresentar indisponível sem preencher valores fictícios.

Exceções

- Timeout, HTTP 429, 5xx ou JSON inválido: retornar indisponível, mantendo detalhe e inscrição.
- Registrar categoria de falha sem dados pessoais; não repetir imediatamente chamadas com erro.

UC04 - Manter locais

Atores: Organizador. **Requisito:** RF02. **Prioridade:** alta.

Objetivo: Manter locais no contexto do Agenda ao Ar Livre.

Precondições: Sessão ativa; para alterar/excluir, ser dono do local.

Pós-condições de sucesso: Local salvo, consultado ou excluído segundo restrições.

Pós-condições de falha: nenhuma alteração parcial; mensagens preservam dados de formulário quando aplicável.

Fluxo principal

1. Abrir lista dos próprios locais e solicitar criação.
2. Informar nome, endereço, cidade, UF, latitude e longitude.
3. Validar campos, limites das coordenadas e contexto de fuso do projeto.
4. Salvar com proprietário extraído da sessão.
5. Permitir edição ou exclusão após verificar referências.

Fluxos alternativos

- Editar: endereço/coordenadas bloqueados quando há atividade publicada ou histórico de inscrição.
- Excluir: permitido apenas sem atividades relacionadas.

Exceções

- Dados inválidos: não salvar e destacar campos.
- Local de outro usuário: negar acesso; referência existente impede exclusão.

UC05 - Manter e publicar atividades

Atores: Organizador. **Requisito:** RF03. **Prioridade:** alta.

Objetivo: Manter e publicar atividades no contexto do Agenda ao Ar Livre.

Precondições: Sessão ativa e local próprio; categorias previamente cadastradas pela administração.

Pós-condições de sucesso: Rascunho salvo, atividade publicada/editada ou rascunho elegível excluído.

Pós-condições de falha: nenhuma alteração parcial; mensagens preservam dados de formulário quando aplicável.

Fluxo principal

1. Abrir formulário e informar título, descrição, local, categoria, início, término e capacidade.
2. Validar RN01-RN04; salvar rascunho.
3. Revisar e solicitar publicação.
4. Publicar e tornar atividade visível no catálogo.
5. Para edição/exclusão, verificar dono, situação, horário e histórico sob bloqueio transacional.

Fluxos alternativos

- Capacidade menor que confirmados: recusar.
- Histórico de inscrição: bloquear mudança de local, categoria e horários.
- Excluir rascunho sem inscrições: confirmar e excluir; atividade publicada deve usar UC09.

Exceções

- Falha de banco: rollback.
- Acesso por não dono ou operação após início: negar alteração.

UC06 - Inscrever-se

Atores: Participante. **Requisito:** RF05. **Prioridade:** alta.

Objetivo: Inscrever-se no contexto do Agenda ao Ar Livre.

Precondições: Sessão ativa; atividade publicada, futura e de outro organizador.

Pós-condições de sucesso: Uma inscrição confirmada existe e ocupa uma vaga.

Pós-condições de falha: nenhuma alteração parcial; mensagens preservam dados de formulário quando aplicável.

Fluxo principal

1. Solicitar inscrição no detalhe.
2. Bloquear linha da atividade em transação.
3. Revalidar situação, início, capacidade e inscrição existente.
4. Criar inscrição ou reativar a cancelada, se houver vaga.
5. Confirmar transação e exibir sucesso e vagas atualizadas.

Fluxos alternativos

- Já confirmada: informar inscrição existente sem duplicar.
- Sem vagas: informar lotação.
- Conta do organizador: recusar autoinscrição.

Exceções

- Concorrência: a segunda requisição vê contagem atualizada após o bloqueio.
- Falha de banco: rollback sem consumir vaga.

UC07 - Consultar minhas inscrições

Atores: Participante. **Requisito:** RF06. **Prioridade:** alta.

Objetivo: Consultar minhas inscrições no contexto do Agenda ao Ar Livre.

Precondições: Sessão ativa.

Pós-condições de sucesso: Lista restrita ao usuário apresentada.

Pós-condições de falha: nenhuma alteração parcial; mensagens preservam dados de formulário quando aplicável.

Fluxo principal

1. Abrir Minhas inscrições.
2. Buscar somente registros do usuário autenticado.
3. Mostrar título, data, situação e motivo de cancelamento quando houver.
4. Oferecer detalhe e cancelamento se elegível.

Fluxos alternativos

- Sem inscrições: apresentar orientação e acesso ao catálogo.

Exceções

- Sessão expirada: solicitar login.
- Não aceitar id de outro participante como filtro.

UC08 - Cancelar inscrição

Atores: Participante. **Requisito:** RF06. **Prioridade:** alta.

Objetivo: Cancelar inscrição no contexto do Agenda ao Ar Livre.

Precondições: Ser dono da inscrição; atividade ainda não iniciada.

Pós-condições de sucesso: Inscrição cancelada pelo participante; vaga liberada se antes confirmada.

Pós-condições de falha: nenhuma alteração parcial; mensagens preservam dados de formulário quando aplicável.

Fluxo principal

1. Solicitar cancelamento e confirmar intenção.
2. Bloquear atividade e localizar inscrição do usuário na transação.
3. Se confirmada e elegível, mudar para cancelada com motivo participante.
4. Salvar cancelada_em e atualizar interface.

Fluxos alternativos

- Já cancelada: informar situação sem liberar vaga novamente.
- Atividade cancelada: manter motivo existente.

Exceções

- Atividade já iniciada: recusar operação.
- Registro de outro usuário: negar acesso.

UC09 - Cancelar atividade

Atores: Organizador. **Requisito:** RF07. **Prioridade:** alta.

Objetivo: Cancelar atividade no contexto do Agenda ao Ar Livre.

Precondições: Ser dono; atividade publicada e futura.

Pós-condições de sucesso: Atividade cancelada e inscrições confirmadas canceladas atomicamente.

Pós-condições de falha: nenhuma alteração parcial; mensagens preservam dados de formulário quando aplicável.

Fluxo principal

1. Solicitar cancelamento na gestão.
2. Informar motivo de 10 a 500 caracteres e confirmar.
3. Bloquear atividade, revalidar dono e início.
4. Salvar situação, motivo e cancelada_em; cancelar inscrições confirmadas com motivo atividade_cancelada.
5. Confirmar transação; exibir informação pública e em Minhas inscrições.

Fluxos alternativos

- Já cancelada: manter resultado anterior sem nova alteração.

Exceções

- Motivo inválido: não salvar.
- Falha ao atualizar inscrições: reverter toda a transação.

UC10 - Gerar e exportar relatório

Atores: Organizador. **Requisito:** RF09. **Prioridade:** alta.

Objetivo: Gerar e exportar relatório no contexto do Agenda ao Ar Livre.

Precondições: Sessão ativa; filtros válidos.

Pós-condições de sucesso: Indicadores exibidos e CSV equivalente disponível.

Pós-condições de falha: nenhuma alteração parcial; mensagens preservam dados de formulário quando aplicável.

Fluxo principal

1. Informar intervalo inclusivo de datas de início e situação opcional.
2. Filtrar somente atividades do organizador.
3. Agrupar inscrições únicas atuais por confirmada/cancelada.
4. Exibir totais de atividades, publicadas, canceladas, inscrições e ocupação.
5. Solicitar CSV, usando os mesmos filtros e regras de acesso.

Fluxos alternativos

- Sem registros: apresentar zeros, ocupação nula e CSV só com cabeçalho.

Exceções

- Intervalo invertido: recusar e indicar campos.
- Sessão expirada: solicitar autenticação; nunca gerar relatório global.

UC11 - Consultar API pública

Atores: Cliente de API (terceiro). **Requisito:** RF10. **Prioridade:** alta.

Objetivo: Consultar API pública no contexto do Agenda ao Ar Livre.

Precondições: Requisição HTTP para rota v1 documentada.

Pós-condições de sucesso: JSON público ou erro HTTP padronizado retornado.

Pós-condições de falha: nenhuma alteração parcial; mensagens preservam dados de formulário quando aplicável.

Fluxo principal

1. Enviar GET ao catálogo com filtros opcionais.
2. Validar parâmetros e aplicar limite de requisições.
3. Consultar os mesmos serviços de leitura usados pela interface.
4. Retornar JSON paginado com status 200; detalhe e previsão usam id da atividade.

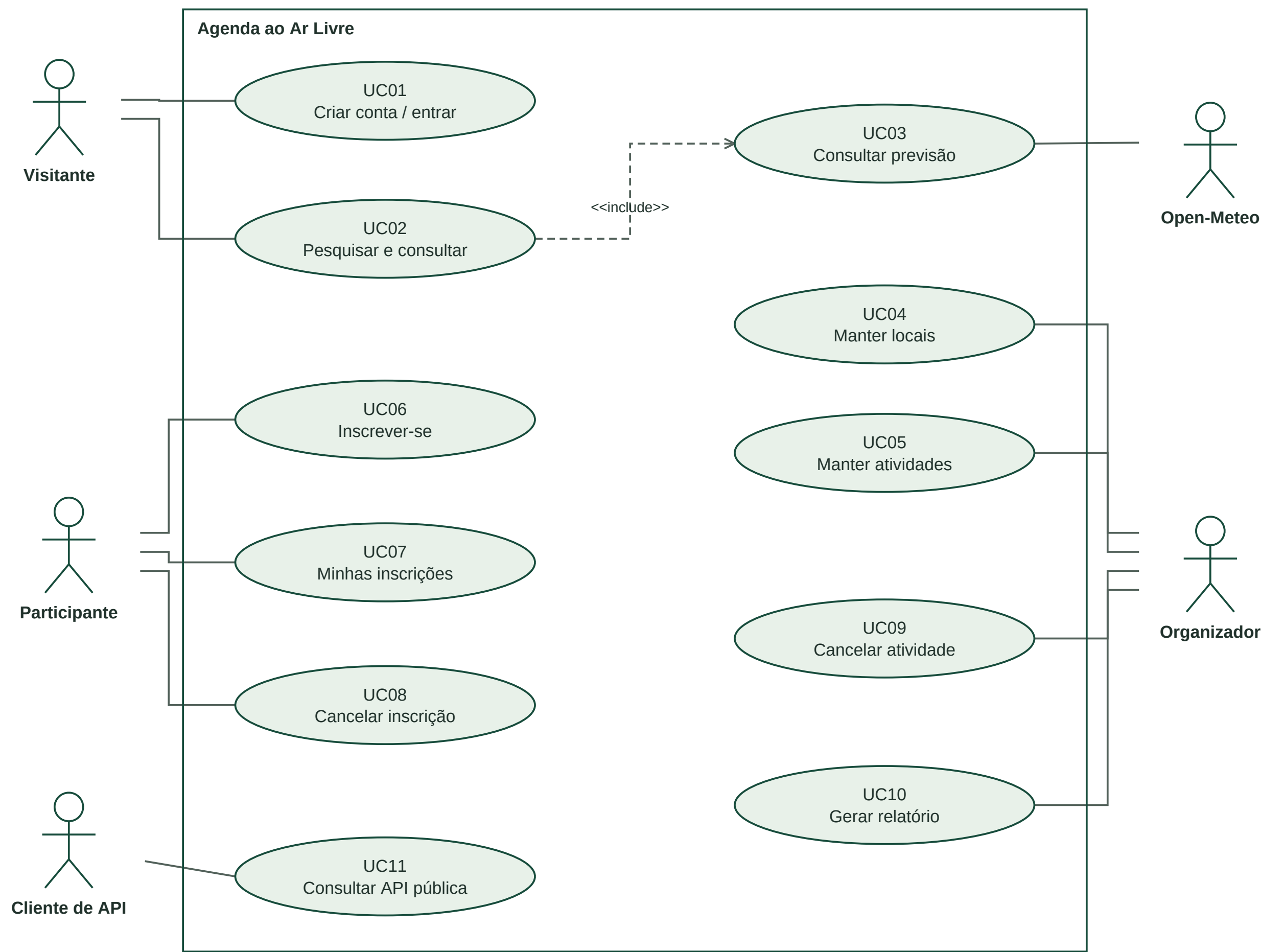
Fluxos alternativos

- GET categorias: listar catálogo de categorias.
- GET previsão: executar UC03; indisponibilidade é estado do recurso com HTTP 200.
- Rascunho: retornar 404.

Exceções

- Parâmetro inválido: 400; rota/id inexistente: 404; método não permitido: 405.
- Limite excedido: 429 com Retry-After; falha interna: 500 sem detalhes sensíveis.

Diagrama UML de casos de uso



Arquitetura da aplicação

Agenda ao Ar Livre | Proposta para implementação na fase 2

1. Decisão principal

Monólito modular Django com templates renderizados no servidor e uma API REST de leitura em Django REST Framework. Interface e API reutilizam regras e consultas do domínio; a interface não precisa chamar a API pública para realizar escritas. Isso reduz duplicação e evita implementar uma SPA para uma equipe de duas pessoas.

Python 3.12 e Django 5.2 LTS são a linha de base proposta; versão corretiva e versões compatíveis de DRF, psycopg, requests, Gunicorn e WhiteNoise serão fixadas e verificadas ao iniciar a fase 2. PostgreSQL será usado em desenvolvimento e produção para reproduzir bloqueios e concorrência. Não existe aplicação Django implementada nesta entrega.

2. Componentes UML

Diagrama ou prancha visual disponível ao final deste documento; fonte editável e SVG na mesma pasta.

Fonte editável: componentes.drawio. Componentes UML estão marcados com estereótipo component e suas dependências são direcionadas. A Open-Meteo está fora do limite da aplicação.

3. Camadas e responsabilidades

Componente	Responsabilidade
Navegador / templates	Formulários, catálogo, detalhe, feedback, inscrições e relatório.
contas	Usuário customizado AbstractUser com e-mail único; sessão, login e logout.
agenda	Models Local, Categoria, Atividade e Inscricao; formulários e views.
Serviços de domínio	Publicar, inscrever, cancelar e autorizar sob transações.
relatorios	Consultas agregadas restritas ao organizador e exportação CSV.
api	Serializers públicos, paginação, filtros, normalização de erros e rotas /api/v1/.
meteorologia	Cliente HTTP com host fixo, validação JSON e normalização dos estados.
ORM / PostgreSQL	Persistência, chaves, checks, unicidade e bloqueios de linha.
Cache	Backend DatabaseCache Django com TTL, incluindo cache negativo breve.

DatabaseCache usa tabela técnica própria criada pelo Django; não é entidade de negócio. Também existem tabelas técnicas de auth, sessões, permissões, migrations e, se habilitado, admin log. Elas não são omitidas da implantação, apenas separadas do DER de domínio.

4. Fluxos de dados

Consulta: navegador -> URL/view -> serviço de consulta -> ORM -> PostgreSQL -> template HTML. API: cliente -> DRF -> mesmo serviço -> serializrer com lista explícita de campos -> JSON.

Inscrição: sessão + CSRF -> view -> serviço -> transaction.atomic + select_for_update na atividade -> verificação de início/situação/vagas -> criação ou reativação -> commit -> mensagem. Cancelamentos e edição de capacidade usam o mesmo bloqueio, evitando decisões com contagem antiga.

Previsão: página de detalhe -> serviço meteorológico -> horizonte/cache -> HTTPS Open-Meteo quando necessário -> validação -> cache -> bloco visual. A chamada não ocorre dentro da transação de inscrição. Falhas do provedor não bloqueiam o negócio.

5. Organização prevista de código

Usar src/backend/ para manage.py, config/ e apps contas/, agenda/, meteorologia/, relatorios/ e api/. Templates e static ficam no projeto Django. A fase 1 entrega apenas os documentos e protótipos, evitando instruções de execução de código inexistente.

6. Autenticação e autorização

Contas são participantes e podem organizar atividades próprias. Dono vem sempre da sessão, nunca de um campo enviado pelo formulário. Querysets protegidos filtram por dono antes da consulta de objeto. Operações POST exigem CSRF. API v1 é exclusivamente GET, pública; não possui endpoints de escrita nem token nesta versão. GET nunca muda inscrições.

Senhas usam mecanismos do Django; USERNAME_FIELD será email no usuário customizado desde a primeira migration. Sessão expira conforme configuração a definir na implementação. Middleware de limitação de login deverá ser configurado e testado; DRF throttle para a API é defesa complementar, não garantia contra ataque distribuído.

7. Implantação prevista

Cliente -> HTTPS do provedor -> Unicorn/Django -> PostgreSQL gerenciado. WhiteNoise serve arquivos estáticos coletados. O backend acessa Open-Meteo por HTTPS de saída. Não há upload de arquivos no escopo.

Provedor e domínio ainda não escolhidos: avaliar suporte a Python/PostgreSQL, custo, persistência, logs e disponibilidade até o fim da avaliação. A escolha ocorrerá no marco M2, sem contratação nesta fase.

DEBUG=False, ALLOWED_HOSTS restrito, CSRF_TRUSTED_ORIGINS com origem HTTPS exata, cookies Secure/HttpOnly conforme tipo e configuração correta do proxy confiável. Credenciais via ambiente; .env não versionado. Backup do banco e restauração de dados fictícios planejados para demonstração.

8. Decisões e consequências

- Templates Django: menos ferramentas e deploy único; evolução de interface sem SPA.
- PostgreSQL também local: configuração inicial adicional, mas teste de última vaga reproduz produção.
- API somente leitura: atende consulta por terceiros sem ampliar manutenção e autenticação de API.
- Previsão diária: minimiza interpretação errada de valores horários; interface identifica que o resumo se refere ao dia inteiro.
- Sem Celery/Redis: consulta sob demanda com cache de banco; adequada ao volume acadêmico.

9. Segurança e validação planejadas

Testar IDOR entre dois organizadores, CSRF, unicidade, XSS por título, filtros inválidos, exportação CSV com neutralização de células iniciadas por =,+,-,@, concorrência da última vaga e falhas externas. Planejar SAST com Bandit, análise de dependências complementar e DAST com OWASP ZAP apenas em ambiente do grupo. Relatórios e correções serão produzidos na fase 2, sem declarar execução antecipada.

Arquitetura UML de componentes

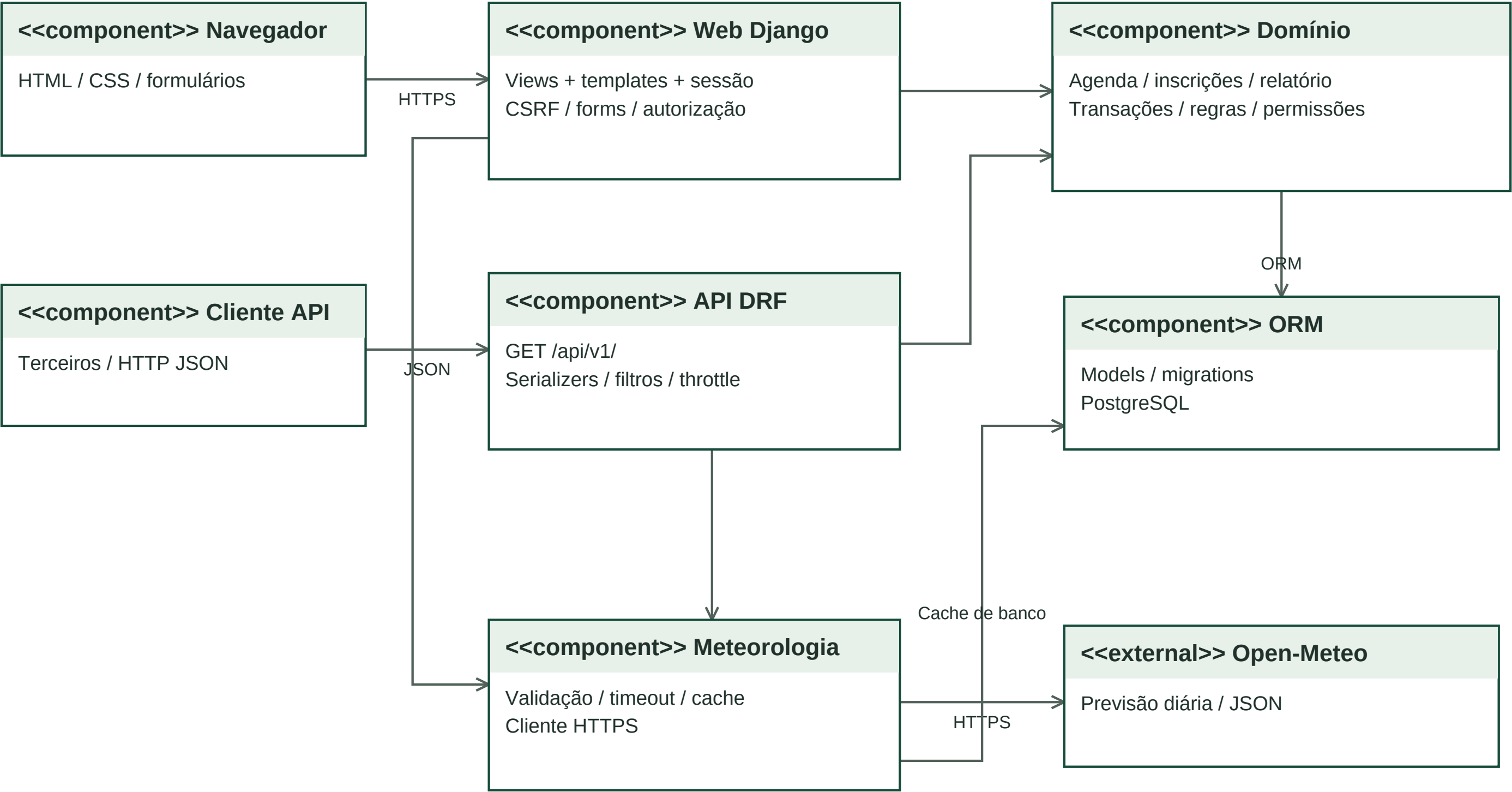


Diagrama de classes

Agenda ao Ar Livre | Modelo estrutural proposto

Diagrama ou prancha visual disponível ao final deste documento; fonte editável e SVG na mesma pasta.

Fonte editável: diagrama-de-classes.drawio.

Responsabilidades

Usuario representa identidade; Local descreve um espaço sob gestão de um organizador; Categoria classifica a atividade; Atividade concentra situação, período e capacidade; Inscricao resolve o vínculo entre participante e atividade.

Operações inscrever, cancelar e publicar ficam em serviços transacionais, e não em controllers duplicados. Relatório e previsão são resultados derivados e não entidades persistidas de negócio. O cache meteorológico é infraestrutura.

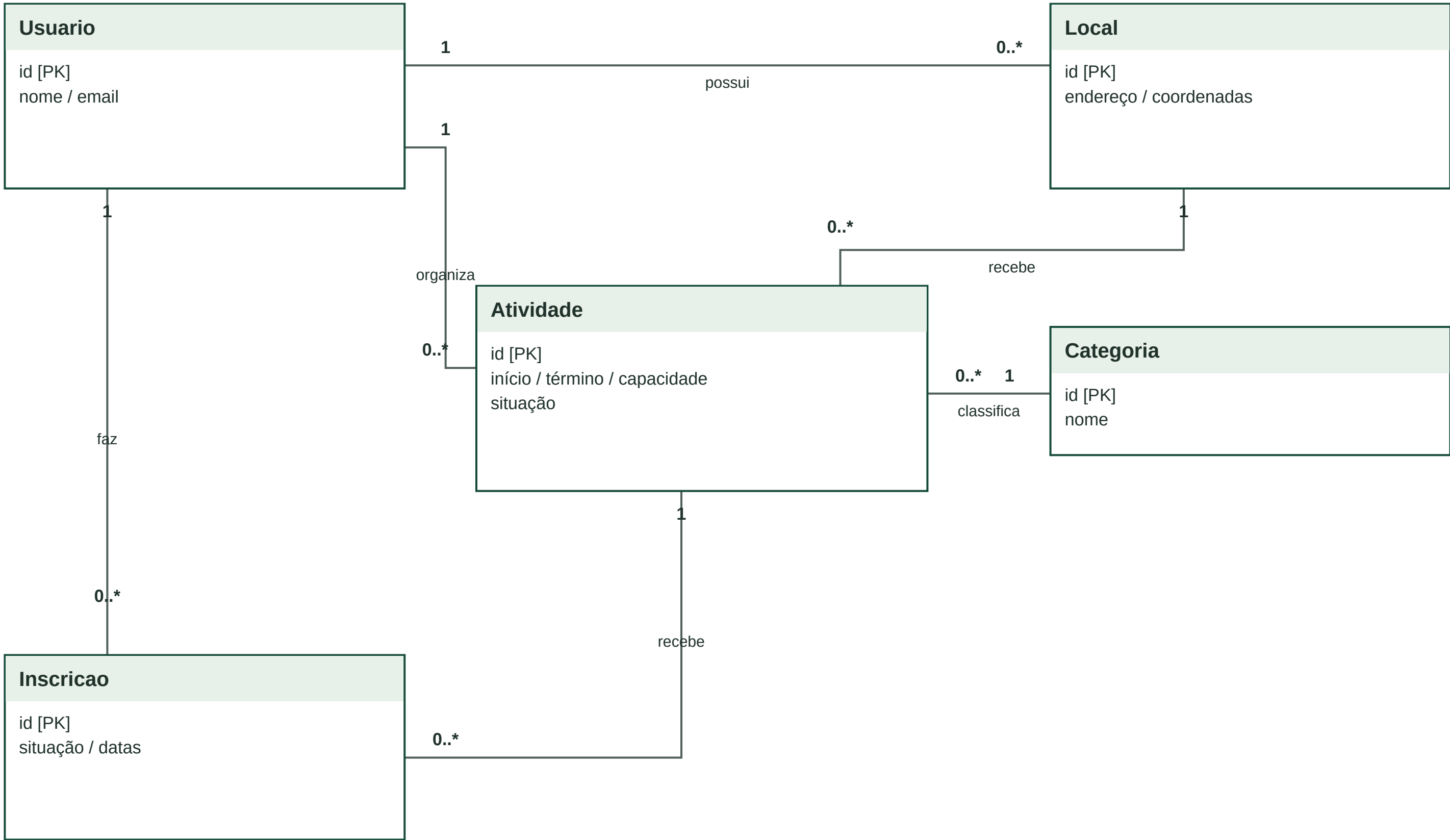
Multiplicidades

Um Usuario possui 0..* locais e organiza 0..* atividades; cada local e atividade tem exatamente 1 dono. Um Local recebe 0..* atividades; cada atividade ocorre em 1 local. Uma Categoria classifica 0..* atividades; cada atividade tem 1 categoria. Um Usuario possui 0..* inscrições; cada inscrição pertence a 1 usuário e 1 atividade; uma atividade possui 0..* inscrições.

Integridade

Inscricao é única pelo par usuario_id/atividade_id. Capacidade não é lista de inscrições: vagas são calculadas. Relações não usam composição UML, pois a exclusão de objetos com histórico é protegida.

Diagrama UML de classes



Modelo entidade-relacionamento

Agenda ao Ar Livre | Domínio de negócio

Diagrama ou prancha visual disponível ao final deste documento; fonte editável e SVG na mesma pasta.

Fonte editável: diagrama-er.drawio. A notação mostra entidades e cardinalidades mínimo..máximo nas extremidades. Os atributos completos e tipos constam no modelo lógico complementar.

Entidades e relacionamentos

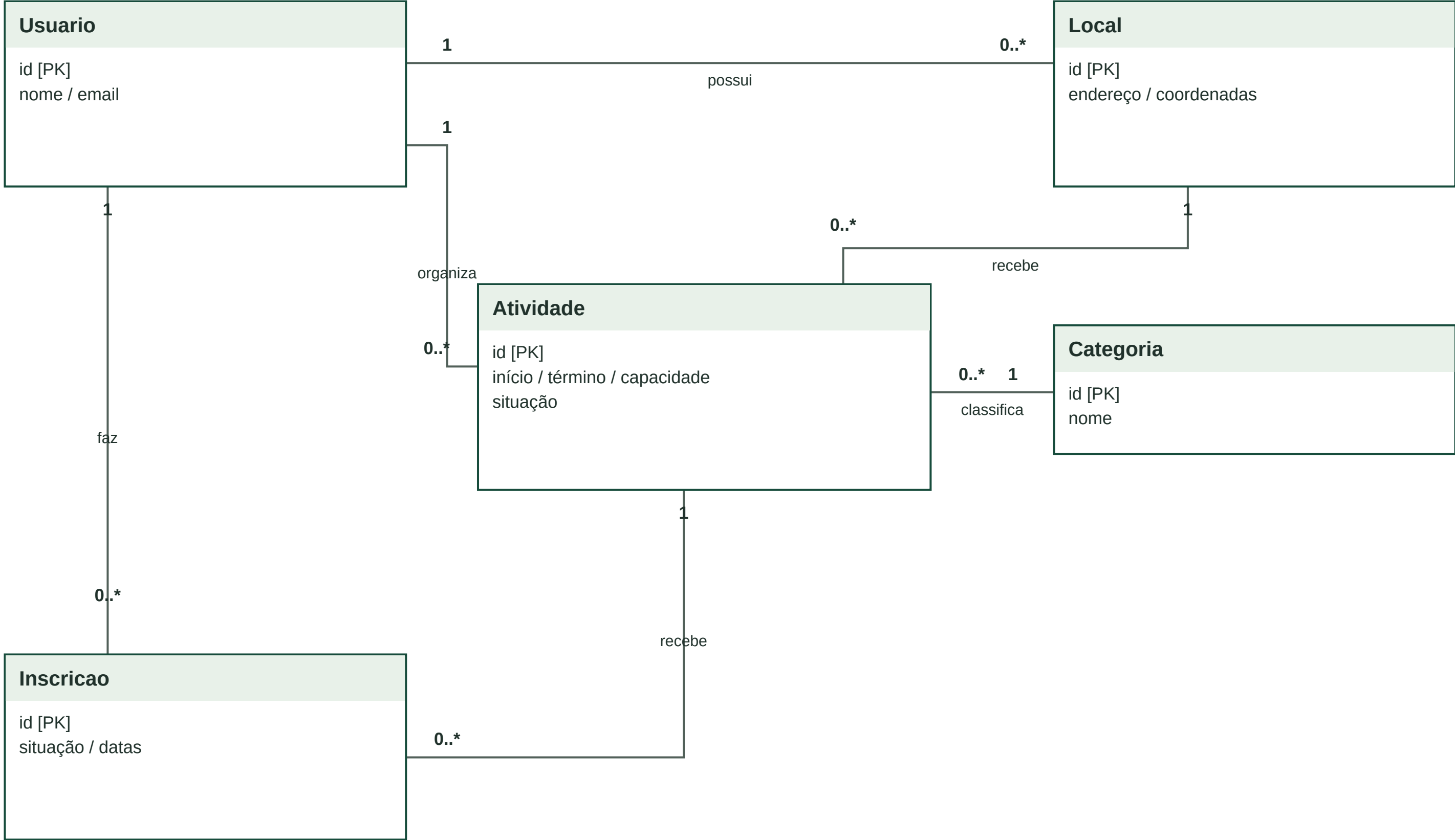
Relacionamento	Cardinalidade e significado
Usuario possui Local	Usuario 1 -> Local 0..*. Cada local tem um dono.
Usuario organiza Atividade	Usuario 1 -> Atividade 0..*. Toda atividade tem organizador.
Local recebe Atividade	Local 1 -> Atividade 0..*. Toda atividade ocorre em um local.
Categoria classifica Atividade	Categoria 1 -> Atividade 0..*. Toda atividade pertence a uma categoria.
Usuario faz Inscricao	Usuario 1 -> Inscricao 0..*. Toda inscrição tem participante.
Atividade recebe Inscricao	Atividade 1 -> Inscricao 0..*. Toda inscrição se refere a uma atividade.

O relacionamento N:N entre usuários participantes e atividades é resolvido por Inscricao, que contém situação e datas. O papel organizador pertence à relação de autoria da atividade, não a uma tabela ou perfil exclusivo.

Normalização

Categorias e locais não são repetidos textualmente em Atividade. Inscricao depende do par usuário/atividade e possui identificador próprio. Não armazenar vagas, taxa de ocupação ou indicadores, pois são derivados. O modelo busca a terceira forma normal. As tabelas técnicas do Django são descritas na arquitetura.

Diagrama entidade-relacionamento



Modelo lógico e dicionário de dados

Agenda ao Ar Livre | PostgreSQL | Proposta, não migration executada

Diagrama ou prancha visual disponível ao final deste documento; fonte editável e SVG na mesma pasta.

Fonte editável: modelo-logico.drawio. Fonte estruturada: modelo.json. Todos os campos são NOT NULL, salvo last_login e os campos cancelada_em. Campos de motivo aceitam string vazia somente nos estados sem cancelamento.

Usuario

Campo	Tipo	Restrição / significado
id	bigint	PK; identity
email	varchar(254)	UNIQUE; minúsculas; obrigatório
nome	varchar(150)	obrigatório
password	varchar(128)	hash Django; obrigatório
is_active	boolean	default true
is_staff	boolean	default false
is_superuser	boolean	default false
date_joined	timestampz	obrigatório
last_login	timestampz	pode ser nulo

Local

Campo	Tipo	Restrição / significado
id	bigint	PK; identity
proprietario_id	bigint	FK Usuario; PROTECT
nome	varchar(120)	obrigatório
endereço	varchar(255)	obrigatório
cidade	varchar(120)	obrigatório
uf	char(2)	UF brasileira válida
latitude	numeric(9,6)	entre -90 e 90
longitude	numeric(9,6)	entre -180 e 180
criado_em	timestampz	obrigatório

Categoria

Campo	Tipo	Restrição / significado
id	bigint	PK; identity
nome	varchar(60)	UNIQUE; obrigatório

Atividade

Campo	Tipo	Restrição / significado
id	bigint	PK; identity
organizador_id	bigint	FK Usuario; PROTECT
local_id	bigint	FK Local; PROTECT
categoria_id	bigint	FK Categoria; PROTECT
titulo	varchar(120)	obrigatório
descricao	text	10 a 5000 caracteres
inicio	timestampz	obrigatório
termino	timestampz	maior que inicio
capacidade	integer	entre 1 e 500
situacao	varchar(12)	rascunho/publicada/cancelada
motivo_cancelamento	varchar(500)	vazio salvo se cancelada
cancelada_em	timestampz	nulo salvo se cancelada
criado_em	timestampz	obrigatório
atualizado_em	timestampz	obrigatório

Inscricao

Campo	Tipo	Restrição / significado
id	bigint	PK; identity
usuario_id	bigint	FK Usuario; PROTECT
atividade_id	bigint	FK Atividade; PROTECT
situacao	varchar(12)	confirmada/cancelada
motivo_cancelamento	varchar(20)	vazio/participante/atividade_cancelada
criado_em	timestampz	primeira inscrição
atualizado_em	timestampz	última mudança
cancelada_em	timestampz	nulo se confirmada

Restrições complementares

- **Usuario:** usuário customizado baseado em `AbstractUser`; remover `username/first_name/last_name` e usar `nome/email`. Definir antes da primeira migration. Relações técnicas `groups/user_permissions` permanecem conforme Django.
- **Email:** normalizar para minúsculas e impor unicidade de `Lower(email)` no PostgreSQL; não depender apenas do formulário.
- **Inscricao:** `UNIQUE(usuario_id, atividade_id)`. Reativação limpa `cancelada_em` e motivo, preservando `criado_em` e atualizando `atualizado_em`.
- **Atividade:** `CHECK termino > inicio`, capacidade `BETWEEN 1 AND 500` e situação no conjunto permitido. Cancelada exige `cancelada_em` e motivo com 10 a 500 caracteres; outros estados exigem `cancelada_em` nulo e motivo vazio.
- **Inscricao:** `CHECK` situação permitida; confirmada exige `cancelada_em` nulo e motivo vazio; cancelada exige data e motivo participante ou `atividade_cancelada`.
- **Local:** `CHECK` latitude e longitude nos intervalos; validar UF contra lista permitida. Validações textuais impedem apenas espaços.
- Todos os FKs usam `PROTECT` no ORM; não há exclusão em cascata de histórico. O esquema SQL ilustrativo usa `ON DELETE RESTRICT`.
- Regras entre tabelas (dono do local, capacidade versus contagem, autoinscrição e horários) ficam no serviço transacional, pois `CHECK` não consulta outras linhas.
- Mesmo dia e início futuro são validados na aplicação com fuso `America/Sao_Paulo`; timestamps são guardados com consciência de fuso (UTC na aplicação).

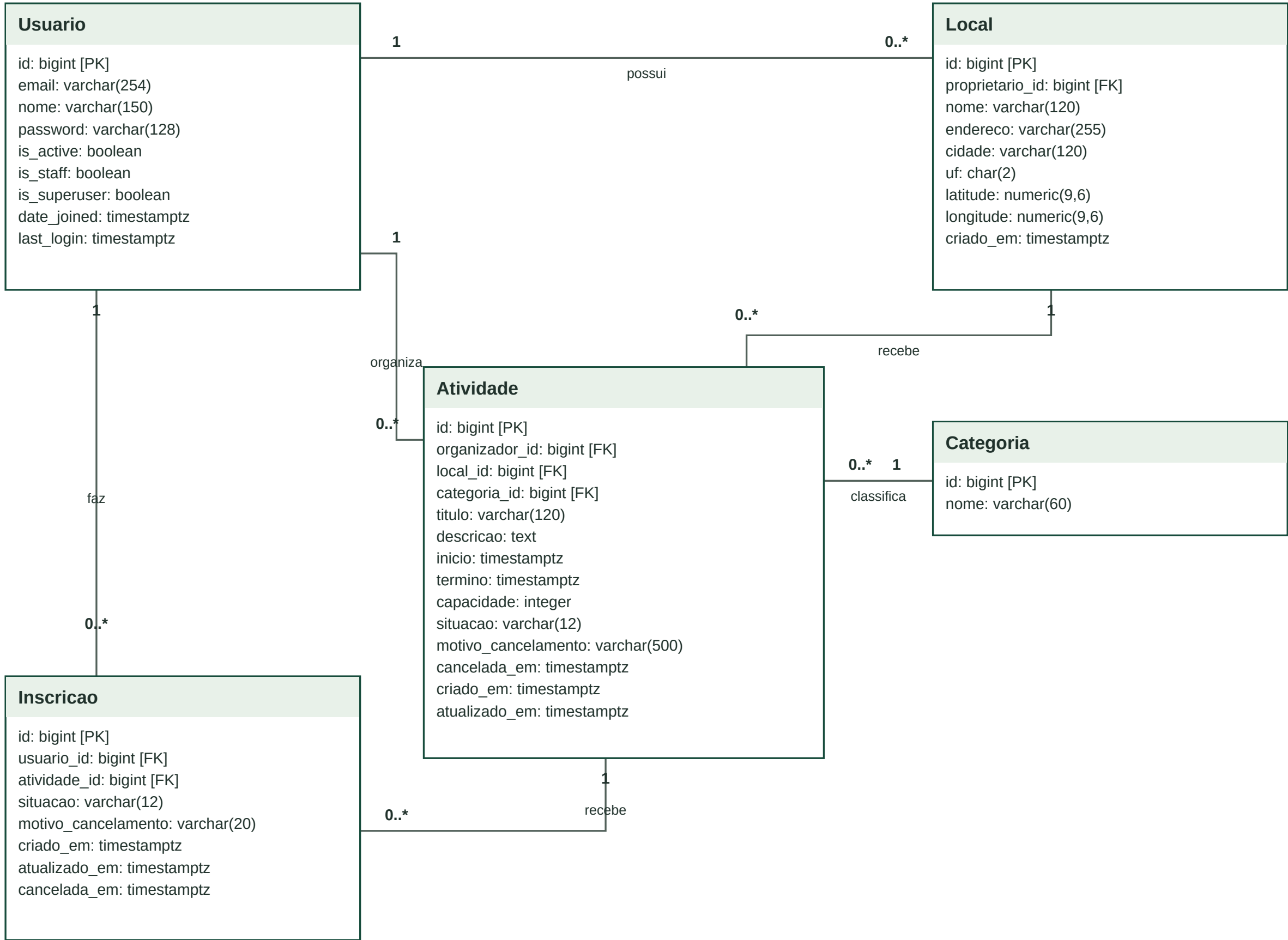
Índices

FKs indexadas pelo Django. Acrescentar `Atividade(situacao, inicio, id)`, `Atividade(organizador_id, inicio)`, `Local(cidade)`, `Inscricao(atividade_id, situacao)`. O índice único do par usuário/atividade auxilia consulta individual. Busca textual inicial com `icontains`; índice trigram fica fora do escopo até medir necessidade.

Migrações e dados iniciais

Usar migrations Django como fonte executável na fase 2; o arquivo `esquema.sql` é referência de domínio, não substitui tabelas técnicas ou migrations. Categorias iniciais: Caminhada, Treino e Encontro, inseridas por data migration. Dados fictícios de demonstração devem ser criados por comando próprio e nunca incluir senhas reais.

Modelo lógico - tabelas e chaves



Contrato inicial da API REST

Agenda ao Ar Livre | v1 | Proposta para fase 2

Escopo e autenticação

API própria pública de consulta, implementada com Django REST Framework. Base local prevista: `http://localhost:8000/api/v1/`. URL de produção ainda não definida. Arquivo legível por ferramentas: `openapi.json` (OpenAPI 3.0.3).

Não é necessário login ou token para GET. Escritas de conta, local, atividade e inscrição serão feitas pelas views web com sessão e CSRF; elas não são rotas da API pública v1. Essa decisão atende ao requisito de consulta por terceiros e reduz exposição.

Endpoints

Método e rota	Finalidade
GET /atividades/	Catálogo paginado; exclui rascunhos, inclui publicada/cancelada.
GET /atividades/{id}/	Detalhe público, sem dados pessoais de participantes.
GET /atividades/{id}/previsao/	Resumo diário ou estado fora_do_horizonte/indisponível.
GET /categorias/	Lista de categorias, ordenada por nome; sem paginação nesta versão.

Paths relativos à base `/api/v1/`. Barras finais fazem parte do contrato. Métodos de escrita retornam 405; HEAD e OPTIONS seguem suporte padrão do framework.

Filtros e paginação

Catálogo: `q` (texto até 120 caracteres em título/descrição), `cidade` (texto até 120, correspondência sem diferenciar maiúsculas), `categoria` (id inteiro positivo), `data_inicio` e `data_fim` (YYYY-MM-DD, limites inclusivos sobre a data de início em America/Sao_Paulo), `situacao` (publicada ou cancelada), `page` (≥ 1), `page_size` (1..100, padrão 20). Filtros combinados por AND. Sem datas, inclui eventos passados e futuros; sem situação, inclui ambas as públicas. Ordenação fixa: `inicio`, `id`.

Data inicial maior que final ou formato inválido: 400. Categoria válida sem resultados: lista vazia. Página positiva além da última: 404, exceto `page=1` sem registros, que retorna lista vazia. Parâmetro desconhecido: 400 para evitar consultas silenciosamente incorretas.

Exemplo de consulta

```
GET /api/v1/atividades/?cidade=São%20Paulo&data_inicio=2026-10-01&data_fim=2026-10-31&page=1&page_size=20
Accept: application/json
```

```
{"count":1,"next":null,"previous":null,"results":[{"id":1,"titulo":"Caminhada no parque","descricao":"Encontro gratuito para caminhada leve.","inicio":"2026-10-
```

```
10T08:00:00-03:00", "termino": "2026-10-10T10:00:00-03:00", "capacidade": 20, "vagas": 8, "situacao": "publicada", "encerrada": false, "motivo_cancelamento": "", "categoria": {"id": 1, "nome": "Caminhada"}, "local": {"id": 1, "nome": "Parque Ibirapuera", "endereco": "Av. Pedro Álvares Cabral, portão de encontro a confirmar", "cidade": "São Paulo", "uf": "SP", "latitude": -23.5874, "longitude": -46.6576}}}]}
```

Dados acima são exemplos fictícios, não evento real. O detalhe retorna o mesmo objeto de results, sem envelope. next/previous são URLs ou null. Datas incluem offset; encerrada deriva de termino <= agora. Não serializar organizador, e-mail, senha, lista de inscritos nem campos técnicos.

Exemplo de previsão normalizada

```
{ "atividade_id": 1, "data": "2026-10-10", "estado": "disponivel", "min_c": 17.0, "max_c": 26.0, "probabilidade_chuva_pct": 30, "codigo_wmo": 2, "descricao": "Parcialmente nublado", "consultado_em": "2026-10-04T14:00:00-03:00", "fonte": "Open-Meteo", "url_fonte": "https://open-meteo.com/", "mensagem": "Resumo do dia inteiro; não representa apenas o horário da atividade." }
```

Nos estados indisponível ou fora_do_horizonte, todos os campos meteorológicos e consultado_em são null; mensagem explica o motivo. Mantêm-se atividade_id, data, fonte e url_fonte. A resposta é 200 porque o estado da previsão foi consultado corretamente. Atividade inexistente ou rascunho retorna 404.

Erros e códigos HTTP

Código	Significado
200	Consulta concluída, inclusive lista vazia ou previsão indisponível.
400	Filtros, formato, parâmetros desconhecidos ou valores inválidos.
404	Atividade pública inexistente, id inacessível ou página fora do intervalo.
405	Método não aceito; incluir cabeçalho Allow.
429	Limite de consulta excedido; incluir Retry-After em segundos.
500	Falha interna genérica; não expor traceback.

```
{ "erro": { "codigo": "parametros_invalidos", "mensagem": "Revise os filtros informados.", "campos": { "data_fim": [ "Deve ser igual ou posterior à data inicial." ] } } }
```

Outros códigos de erro: nao_encontrado, metodo_nao_permitido, limite_excedido, erro_interno. campos é um objeto vazio quando não há erro de campo. Implementar exception handler DRF para esse envelope; não presumir que seja formato padrão do framework.

Condições de uso

Limite inicial proposto: 60 requisições/minuto por IP, ajustável no ambiente. Não há SLA. Usar cache local no cliente e respeitar Retry-After. Content-Type application/json; UTF-8; não configurar CORS amplo por padrão. CORS não impede clientes servidor-servidor. Um cliente web de outro domínio exigirá decisão explícita na fase 2.

Verificação futura

Importar openapi.json em ferramenta compatível; verificar catálogo, filtros, paginação, rascunho inacessível, erro uniforme, limite e indisponibilidade meteorológica. Os endpoints estão especificados, ainda não implementados.

Plano de integração externa

Agenda ao Ar Livre | Open-Meteo | Consulta documental em 04/10/2026

Finalidade

Apresentar resumo meteorológico diário na página da atividade e pela API própria. O organizador usa essa informação para decidir manualmente se mantém ou cancela a atividade. Participantes consultam o mesmo resumo para se planejar. Não calcular previsão nem inventar dados ausentes.

Serviço e documentação

Provedor: Open-Meteo. Endpoint HTTPS: <https://api.open-meteo.com/v1/forecast>. Documentação: <https://open-meteo.com/en/docs>. Termos: <https://open-meteo.com/en/terms>. Acesso gratuito não comercial sem chave; reavaliar condições antes da publicação e atribuir a fonte com link em toda exibição.

A documentação permite solicitar até 16 dias (padrão: 7). A implementação adotará deliberadamente horizonte conservador de 7 dias, incluindo hoje (hoje a hoje+6), verificando se a data efetivamente veio na resposta. Fora dessa janela retorna `fora_do_horizonte` sem chamada. Atividades passadas não usam previsão histórica.

Requisição planejada

GET `/v1/forecast` com latitude e longitude do Local, `daily=temperature_2m_min,temperature_2m_max,precipitation_probability_max,weather_code`, `timezone=America/Sao_Paulo`, `forecast_days=7`, `temperature_unit=celsius`.

Exemplo ilustrativo de coordenadas de local público, não localização de usuário:

```
https://api.open-meteo.com/v1/forecast?latitude=-23.5874&longitude=-46.6576&daily=temperature_2m_min,temperature_2m_max,precipitation_probability_max,weather_code&timezone=America%2FSao_Paulo&forecast_days=7&temperature_unit=celsius
```

Dados utilizados

Campo externo	Uso no sistema
<code>daily.time</code>	Selecionar exatamente a data local da atividade.
<code>temperature_2m_min / max</code>	Exibir mínima e máxima diária em graus Celsius.
<code>precipitation_probability_max</code>	Exibir probabilidade máxima diária de precipitação, em porcentagem.
<code>weather_code</code>	Traduzir código WMO para condição; código desconhecido produz descrição não disponível.
<code>daily_units</code>	Validar unidades esperadas antes de exibir.

Os valores correspondem ao dia inteiro, não especificamente ao horário do encontro. `consultado_em` é o horário de obtenção pelo nosso servidor, não o horário de geração do modelo meteorológico. Respostas de

exemplo nos documentos e protótipos são fictícias.

Robustez

1. Validar coordenadas no cadastro e usar URL fixa do provedor, nunca URL informada pelo cliente.
2. Consultar cache por latitude, longitude, data e fuso; TTL de 1.800 segundos.
3. Em falta de cache, usar timeout de conexão de 2 s e leitura de 3 s; sem retry automático no mesmo pedido. Tempo total alvo 5 s; configurar também prazo de resposta no servidor.
4. Validar HTTP, JSON, arrays de mesmo tamanho, data desejada, unidades e números finitos; rejeitar mínimo maior que máximo e probabilidade fora de 0..100.
5. Valores nulos ou incompletos geram indisponível; manter a página e todas as operações de inscrição acessíveis.
6. Erros 429 respeitam Retry-After quando válido; aplicar cache negativo de pelo menos 60 s, sem dormir na requisição. Timeout/5xx/JSON inválido também usam 60 s de cache negativo.
7. Cache expirado não é apresentado como previsão atual; sem resposta nova, mostrar indisponível.
8. Registrar tipo de erro e duração; não registrar credenciais ou dados de participantes.

Limites e licença

Na consulta de 04/10/2026, os termos gratuitos indicam menos de 10.000 chamadas/dia, 5.000/hora e 600/minuto; a tabela também indica 300.000/mês. Uso não comercial e atribuição CC BY 4.0. O projeto adotará orçamento operacional inferior aos limites, com cache e contagem de chamadas; revisar termos no deploy.

Atribuição na interface: Dados meteorológicos: Open-Meteo (CC BY 4.0), com links ao provedor e à licença <https://creativecommons.org/licenses/by/4.0/>. O resumo é uma transformação dos campos originais.

Testes planejados

Resposta válida; data ausente; horizonte excedido; JSON inválido; campo nulo; arrays inconsistentes; timeout; 429; 5xx; cache hit; expiração. Usar mocks determinísticos para erros e uma demonstração real da integração na fase 2. Esta fase verificou documentação, não comprova execução do serviço integrado.

Identidade visual e protótipos

Agenda ao Ar Livre | Fase 1

Conceito

Uma agenda para encontrar pessoas e ocupar espaços abertos. Assinatura: "Seu próximo encontro começa lá fora." O símbolo original combina montanha/trilha e sol dentro de um calendário; SVG vetorial editável em `images/logo.svg`. Não utiliza imagens externas ou marca de terceiros.

Paleta e tipografia

Token	Cor e uso
Verde principal	#174C3C: títulos, botões e navegação.
Verde suave	#E8F1E9: estados positivos e superfícies.
Fundo	#F7F6F0: base da página.
Texto	#20312B: texto principal.
Texto secundário	#52615A: metadados.
Acento solar	#F2C66D: ilustrações e destaque com texto escuro.
Erro	#A12828: borda e texto de falha, sempre com mensagem.

Tipografia: pilha `system-ui`, `-apple-system`, `Segoe UI`, `sans-serif`; sem download externo de fontes. Base 16 px, títulos 28-48 px, espaçamento múltiplo de 4/8 px, cantos de 12-20 px. Foco visível e botões com altura mínima 44 px. Cor não substitui rótulos.

Telas essenciais

Tela	Casos / propósito
Explorar	UC02: filtros, cartões e resultado vazio.
Detalhe	UC02-UC03-UC06: informações, previsão diária, vagas e inscrição.
Entrar / criar conta	UC01: formulário e validação de campos.
Minhas inscrições	UC07-UC08: lista, estado e cancelamento.
Organizar	UC05-UC09: lista de atividades e ações do dono.
Nova atividade	UC05: local, categoria, horários, capacidade e publicação.
Locais	UC04: cadastro de endereço e coordenadas.
Relatórios	UC10: período, situação, indicadores e CSV.

Arquivos e modo de uso

Abrir index.html em navegador. Fontes editáveis: index.html, styles.css e app.js. A galeria telas.md reúne as oito capturas PNG, também incluídas ao final deste PDF. Protótipo com dados fictícios e interações em memória; não implementa autenticação, backend Django, persistência ou acesso real à previsão. Atualizar a página reinicia a demonstração.

O protótipo simula inscrição/cancelamento, navegação, filtros, cadastro de atividade e exportação demonstrativa. Controles de estados permitem visualizar indisponibilidade meteorológica, fora de horizonte e lotação. Não interpretar a simulação como evidência da fase 2.

Estados e acessibilidade

Formulários com labels; feedback em região aria-live; foco visível; contraste adequado; grids empilham no celular. Detalhe diferencia publicada, cancelada e lotada. Previsão diferencia disponível, indisponível e fora do horizonte; botão de inscrição permanece disponível quando apenas a previsão falha.

Erros do servidor planejados devem aparecer próximos aos campos e no resumo. Listas vazias explicam a próxima ação. Exclusão/cancelamento exigem confirmação. As telas de referência apresentam dados fictícios e a mesma identidade visual dos documentos.

PROTÓTIPO · ENTREGA 1 · DADOS FICTÍCIOS · SEM AUTENTICAÇÃO OU PERSISTÊNCIA REAL

 **Agenda ao Ar Livre**

Explorar

Minhas inscrições

Organizar

Relatórios

Entrar

MAIS ENCONTROS. MAIS AR LIVRE.

Seu próximo encontro começa lá fora.

Encontre uma atividade, confira a previsão e reserve seu lugar. O resto é aproveitar o caminho.

Organizar uma atividade



O que você quer fazer?

Caminhada, treino, encontro...

Cidade

São Paulo

Categoria

Todas

▼

Buscar

A partir de

dd/mm/yyyy

📅

Até

dd/mm/yyyy

📅

Encontre seu próximo programa

3 atividades



Caminhada

Caminhada no parque

10/10/2026, 08:00
Parque Ibirapuera · São Paulo

8 vagas livres

Ver atividade



Treino

Treino para começar o dia

11/10/2026, 07:00
Parque Ibirapuera · São Paulo

6 vagas livres

Ver atividade



Encontro

Piquenique e boas conversas

12/10/2026, 15:00
Parque Ibirapuera · São Paulo

16 vagas livres

Ver atividade

Agenda ao Ar Livre · Luca Romariz e Miguel Silva · Desenvolvimento Web · Protótipo acadêmico

Agenda ao Ar Livre | Protótipo da Entrega 1 | Dados fictícios

[← Voltar para atividades](#)

Caminhada

Caminhada no parque

Uma manhã para caminhar com calma e conhecer novas pessoas. Leve água e use roupas confortáveis.

Quando

10/10/2026, 08:00 até 10:00

Onde

Parque Ibirapuera

Endereço

Av. Pedro Álvares Cabral · portão de encontro a confirmar

Participação

Gratuita

Quero participar

Minhas inscrições

8 vagas disponíveis de 20

PREVISÃO PARA O DIA

17° — 26°

Parcialmente nublado

Probabilidade máxima de chuva: 30%

Resumo do dia inteiro, não apenas do horário da atividade.

Consulta ilustrativa: 04/10/2026, 14h.

Dados: [Open-Meteo](#) · [CC BY 4.0](#)

Valores simulados neste protótipo.

Antes de sair

Confira se a atividade continua confirmada. O organizador decide sobre cancelamentos.

Simular estado da tela

Disponível





FAÇA PARTE

Que bom ter você por aqui.

Entre para participar e organizar atividades.

E-mail

voce@exemplo.com

Senha

Entrar na demonstração

Use dados fictícios. Este formulário apenas simula o fluxo de acesso.

Quero criar uma conta



SUA AGENDA

Minhas inscrições

Acompanhe seus próximos encontros e consulte mudanças.

Confirmada

Caminhada no parque

10/10/2026, 08:00 · Parque Ibirapuera

Ver atividade

Cancelar inscrição



ÁREA DO ORGANIZADOR

Vamos colocar a ideia na agenda?

Nova atividade

Meus locais Ver relatório

Minhas atividades

publicada

Treino para começar o dia

11/10/2026, 07:00 · 10 inscrições confirmadas

Editar

Cancelar atividade

A área do organizador é uma simulação de acesso. No sistema final, cada conta verá apenas suas atividades.

PROTÓTIPO • ENTREGA 1 • DADOS FICTÍCIOS • SEM AUTENTICAÇÃO OU PERSISTÊNCIA REAL

 **Agenda ao Ar Livre**

[Explorar](#) [Minhas inscrições](#) [Organizar](#) [Relatórios](#) [Entrar](#)

[← Voltar para gestão](#)

ORGANIZE UM ENCONTRO

Nova atividade

Título

Ex.: Caminhada de domingo

Descrição

Local

Parque Ibirapuera

Categoria

Caminhada

Início

dd/mm/yyyy, --:-- --

Término

dd/mm/yyyy, --:-- --

Número de vagas

20

Salvar como

Rascunho

Horários no fuso de São Paulo. Atividades devem começar e terminar no mesmo dia.

Salvar atividade

Agenda ao Ar Livre • Luca Romariz e Miguel Silva • Desenvolvimento Web • Protótipo acadêmico

Agenda ao Ar Livre | Protótipo da Entrega 1 | Dados fictícios



[← Voltar para gestão](#)

Meus locais

Locais cadastrados

Parque Ibirapuera

São Paulo, SP · Usado por atividades

Endereço protegido por histórico de atividade.

Cadastrar local

Nome

Endereço e ponto de encontro

Cidade

UF

SP

▼

Latitude

Longitude

Use coordenadas de um espaço público no fuso de São Paulo.
A localização dos participantes não é coletada.

Salvar local

VISÃO DO ORGANIZADOR

Um retrato dos seus encontros.

Atividades e inscrições no período. Dados demonstrativos.

De

01/10/2026



Até

31/10/2026



Situação

Todas



Aplicar filtros

Atividades

1

Confirmadas

10

Canceladas

1

Ocupação

63%

Atividades do período

Exportar CSV

Atividade	Situação	Capacidade	Confirmadas	Canceladas
Treino para começar o dia	publicada	16	10	1

Ocupação considera apenas atividades publicadas. Inscrições representam o estado atual de cada vínculo, não presença.

Planejamento e rastreabilidade

Agenda ao Ar Livre | Luca Romariz e Miguel Silva

Situação e marcos

Prazo da Entrega 1 informado pelo grupo: 05/10/2026 às 08h (horário de Brasília). A revisão e a publicação devem ocorrer antes desse horário. O prazo da fase 2 não foi informado: as semanas de M2 a M6 são uma estimativa relativa à semana de entrega da fase 1, a ajustar ao calendário da disciplina.

Marco	Prazo relativo	Resultado verificável
M1 - Fase 1	Até 05/10/2026, 08h	Revisar pacote, publicar repositório derivado do material oficial, contribuições da dupla e tag.
M2 - Base	Semana 2	Django, PostgreSQL, usuário customizado, migrations e escolha de hospedagem.
M3 - Negócio	Semana 3	Locais, atividades, busca e inscrições com concorrência testada.
M4 - Integrações	Semana 4	API REST, meteorologia, relatório e interface responsiva.
M5 - Qualidade	Semana 5	Testes, publicação, SAST/DAST, correções e nova verificação.
M6 - Fase 2	Semana 6	Revisão final, demonstração ensaiada e tag final.

Backlog e responsabilidades propostas

São atribuições sugeridas, não comprovação de trabalho já realizado. Ambos devem revisar o conjunto e produzir contribuições identificáveis em suas contas.

ID	Tarefa / responsável	Aceite e dependência
B01	Visão e requisitos / Luca	RF/RN revisados por Miguel; M1.
B02	Casos de uso / Miguel	Fluxos e exceções alinhados aos RF; depende B01.
B03	Dados e arquitetura / Luca	DER e componentes revisados; depende B01-B02.
B04	Protótipos e identidade / Miguel	Telas e estados em desktop/celular; depende B02.

ID	Tarefa / responsável	Aceite e dependência
B05	Contratos e plano / ambos	API e rastreabilidade consistentes; depende B03-B04.
B06	GitHub e revisão da entrega / ambos	Template, colaboradores, commits reais e tag; depende B01-B05.
B07	Django, auth e banco / Luca	Migrações, login e validações; depende M1.
B08	Locais/atividades e templates / Miguel	CRUD e permissões; depende B07.
B09	Inscrições e concorrência / Luca	Última vaga segura; depende B08.
B10	API e Open-Meteo / Luca	Contrato e falhas testados; depende B08.
B11	Relatório e responsividade / Miguel	Filtros e CSV coerentes; depende B09.
B12	Deploy e testes integrados / ambos	HTTPS e roteiro reproduzível; depende B10-B11.
B13	SAST e correções / Luca	Bandit e análise de dependências, triagem e reteste; depende B12.
B14	DAST e correções / Miguel	ZAP apenas no sistema autorizado, triagem e reteste; depende B12.
B15	Apresentação / ambos	Demonstração e explicação individual; depende B13-B14.

Rastreabilidade

Requisito	Casos de uso / dados	Evidência prevista
RF01	UC01 / Usuario	Tela acesso; testes de sessão e e-mail único.
RF02	UC04 / Local, Usuario	Tela locais; validações e PROTECT.
RF03	UC05 / Atividade, Local, Categoria	Formulário/gestão; testes de autorização.
RF04	UC02 / Atividade, Local, Categoria	Catálogo e GET /atividades/.
RF05	UC06 / Inscrição, Atividade	Detalhe; teste PostgreSQL concorrente.
RF06	UC07-UC08 / Inscrição	Minhas inscrições; isolamento entre contas.
RF07	UC09 / Atividade, Inscrição	Gestão/cancelamento transacional.
RF08	UC03 / Local, Atividade, cache técnico	Detalhe; GET /atividades/{id}/previsao/.
RF09	UC10 / Atividade, Inscrição	Relatório restrito e CSV.
RF10	UC11 / Atividade, Local, Categoria	OpenAPI; catálogo/detalhe/categorias/previsão.
RF11	UC01-UC10 / apresentação	Protótipos e teste em 360 px.

Fórmulas do relatório

Filtro considera a data local de inicio da atividade, inclusive nos dois limites, e situação opcional. Toda consulta é limitada ao organizador da sessão.

Total de atividades = quantidade de atividades filtradas. Publicadas/canceladas = contagens por situação persistida (publicadas encerradas continuam publicadas). Inscrições confirmadas/canceladas = registros únicos atuais ligados às atividades filtradas. Não mede presença nem histórico de tentativas.

Ocupação = $100 * \text{confirmadas em atividades publicadas filtradas} / \text{soma da capacidade dessas atividades}$. Denominador zero produz null, exibido como "não se aplica". Rascunhos e canceladas não entram no denominador. Exportar uma linha por atividade com id, título, início, situação, capacidade, confirmadas, canceladas e ocupação (vazia para não publicadas). Totais na tela derivam dessas linhas. Neutralizar fórmulas em campos textuais do CSV.

Estratégia de execução

Usar tarefas pequenas, branches docs/ ou feat/, revisão do outro integrante e integração frequente. Atualizar documentos quando mudar regra ou escopo; registrar motivo no histórico. Priorizar fluxo completo: cadastro -> atividade -> busca -> inscrição -> cancelamento -> relatório; depois consolidar API e estados de falha.

Riscos e resposta

Risco	Responsável e ação
Prazo insuficiente	Ambos: concluir revisão/publicação antes de 05/10/2026, 08h; ajustar marcos da fase 2 quando informada.
API externa falha	Luca: cache, timeout, mocks e demonstração com data dentro da janela.
Interface inconsistente	Miguel: revisar protótipos e fluxo no celular.
Vazamento entre contas	Ambos: testar acesso com dois usuários e ids trocados.
Hospedagem suspensa	Luca: monitorar manualmente disponibilidade até fim da avaliação.
Commit concentrado	Ambos: contribuições próprias ao longo do trabalho, sem atribuição fictícia.

Roteiro de validação na fase 2

1. Usuário A cadastra local e atividade; B não consegue editá-los.
2. B pesquisa, consulta previsão e se inscreve; nova tentativa não duplica.
3. Duas requisições disputam última vaga: só uma confirma.
4. B cancela; vaga retorna; reinscrição reativa o registro.
5. A cancela atividade: todas as inscrições confirmadas são canceladas atômicamente.
6. API oculta rascunhos e e-mails; valida filtros e paginação.
7. Previsão falha/fica fora de horizonte: inscrições continuam funcionando.
8. Relatório/CSV mostram mesmos registros, filtrados pelo dono.
9. Validar teclado, 360 px, SAST/DAST, produção sem DEBUG e segredos fora do Git.

Pendências para submissão da fase 1

- Instituição, curso, turma e prazo da Entrega 1 preenchidos. Informar matrículas apenas se exigidas e confirmar o prazo da fase 2.
- Criar repositório GitHub usando o template e adicionar contas corretas da dupla.
- Cada integrante revisar e contribuir com sua conta; este pacote não fabrica autoria individual.
- Verificar acesso do professor e revisar atribuições propostas.
- Registrar URL e hash/tag após publicar; não preencher com o hash do template.